Python Engineering - NumPy Concept Guide

ndim, shape and size: physical dimension vs array dimension

The main confusion comes from using the word **dimension** for two different ideas: the physical/mathematical dimension of a problem and the structural dimension of a NumPy array.

Key mental model: in NumPy, `ndim` tells us how many indices are needed to locate an element.

1. A physical 3D vector can have ndim = 1

```
v = np.array([10.0, 20.0, 30.0])

v.ndim   # 1
v.shape  # (3,)
v.size   # 3
```

Physically the values may represent x, y, z components. NumPy, however, sees one sequence. Only one index is required: v[0], v[1], or v[2].

2. A 3x3 matrix has ndim = 2

```
R = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1]
])

R.ndim   # 2
R.shape  # (3, 3)
R.size   # 9
```

Even if R acts on physical 3D space, the array is a row-column table. Two indices are needed: R[row,column].

shape=(3,3), but **size=9**.

3. Physical x,y,z axes are NOT NumPy axes

```
v = np.array([vx, vy, vz])
# shape=(3,), ndim=1
```

x, y, z carry physical meaning. NumPy axis 0 is a structural direction of the stored data. Three physical components do not imply ndim=3.

4. A 3D rotation matrix still has only two NumPy axes

```
# axis 0 -> rows
# axis 1 -> columns
R.shape == (3,3)
R.ndim  == 2
```

5. Why np.ndim(R) was conceptually wrong in EX20

For a 2D rotation, shape=(2,2) and ndim=2, so np.eye(R.ndim) accidentally works. For a 3D rotation, shape=(3,3) but ndim is still 2.

```
# Not generic:
I = np.eye(R.ndim)

# Correct for a square rotation matrix:
I = np.eye(R.shape[0])
```

6. Dataset of 3D positions

```
positions = np.array([
    [1,2,3], [4,5,6],
    [7,8,9], [10,11,12]
])
```

```
positions.ndim   # 2
positions.shape  # (4,3)
positions.size   # 12
```

axis 0 selects the point; axis 1 selects the x/y/z component.

7. A truly 3D NumPy structure

```
trajectory = np.zeros((100,4,3))

trajectory.ndim   # 3
trajectory.shape  # (100,4,3)
trajectory.size   # 1200

trajectory[time, object, coordinate]
```

Three indices are required, so ndim=3.

8. Mental cheat sheet

Property	Meaning
ndim	Number of NumPy axes / required indices
shape	Length of each axis
size	Total number of elements

Object	ndim	shape	size
scalar 5	0	()	1
physical 3D vector [x,y,z]	1	(3,)	3
3x3 rotation matrix	2	(3,3)	9
100 three-dimensional positions	2	(100,3)	300
100 times x 4 objects x 3 coords	3	(100,4,3)	1200

Final idea

physical dimension != NumPy ndim

A vector may live in physical 3D space while having ndim=1. A matrix acting on that space can have shape=(3,3) and ndim=2. NumPy describes data structure; physics describes the meaning of that data.